

Chinenye Precious Nwachukwu
Super Mart Analytics Capstone
Answers

Section A — Fundamentals
Topics: SELECT, WHERE, DISTINCT,
ORDER BY, LIMIT

Question 1a

Retrieve the first_name, last_name, and email of every customer who lives in Lagos. Sort results alphabetically by last_name, then by first_name.

Answer

```
SELECT first_name, last_name, email
FROM customers
WHERE city = 'Lagos'
ORDER BY last_name ASC, first_name ASC;
```

Question 1b

List the names of all distinct cities to which SuperMart has shipped at least one order. Sort alphabetically.

Answer

```
SELECT DISTINCT shipping_city
FROM orders
ORDER BY shipping_city ASC;
```

Question 1c

Display the top 10 most expensive products by unit_price. Show product_name, category_id, and unit_price, ordered from most to least expensive.

Answer

```
SELECT product_name, category_id, unit_price
FROM products
ORDER BY unit_price DESC
LIMIT 10;
```

Question 1d

List all employees hired on or after 1st January 2021. Display their full name (concatenated as full_name), role, hire_date, and salary, ordered by hire_date ascending.

Answer

```

SELECT
    first_name || ' ' || last_name AS full_name,
    role,
    hire_date,
    salary
FROM employees
WHERE hire_date >= '2021-01-01'
ORDER BY hire_date ASC;

```

Question 1e

Retrieve all orders placed in December across any year. Show order_id, order_date, status, and shipping_city. Order by order_date descending.

Answer

```

SELECT order_id, order_date, status, shipping_city
FROM orders
WHERE EXTRACT(MONTH FROM order_date) = 12
ORDER BY order_date DESC;

```

Section B — Aggregate Functions

Topics: COUNT, SUM, AVG, MIN, MAX, ROUND

Question 2a

How many orders exist for each status? Display the status, the count, and each status as a percentage of all orders, rounded to 2 decimal places. Label the percentage column pct_of_total. Order by count descending.

Answer

```

SELECT
    status,
    COUNT(*) AS count,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM orders), 2) AS pct_of_total
FROM orders
GROUP BY status
ORDER BY count DESC;

```

Question 2b

For each product category, calculate the minimum, maximum, and average unit_price of products in that category. Round the average to 2 decimal places. Display category_name. Order by average price descending.

Answer

```
SELECT
  c.category_name,
  MIN(p.unit_price) AS min_price,
  MAX(p.unit_price) AS max_price,
  ROUND(AVG(p.unit_price), 2) AS avg_price
FROM products p
JOIN categories c ON p.category_id = c.category_id
GROUP BY c.category_name
ORDER BY avg_price DESC;
```

Question 2c

Across all rows in order_items, calculate: total revenue, average revenue per line item, maximum revenue from a single line item, and minimum revenue from a single line item. Round all values to 2 decimal places.

Answer

```
SELECT
  ROUND(SUM(quantity * unit_price * (1 - discount / 100.0)), 2) AS total_revenue,
  ROUND(AVG(quantity * unit_price * (1 - discount / 100.0)), 2) AS avg_line_revenue,
  ROUND(MAX(quantity * unit_price * (1 - discount / 100.0)), 2) AS max_line_revenue,
  ROUND(MIN(quantity * unit_price * (1 - discount / 100.0)), 2) AS min_line_revenue
FROM order_items;
```

Question 2d

How many distinct customers have placed at least one order? What is the average number of orders per ordering customer, rounded to 2 decimal places?

Answer

```
SELECT
  COUNT(DISTINCT customer_id) AS distinct_customers,
  ROUND(COUNT(*) * 1.0 / COUNT(DISTINCT customer_id), 2) AS
avg_orders_per_customer
FROM orders;
```

Section C — Grouping
Topics: GROUP BY · HAVING

Question 3a

Count the number of customers who registered each year between 2018 and 2024. Display the registration year and the count. Order by year ascending.

Answer

```
SELECT
    EXTRACT(YEAR FROM registration_date) AS registration_year,
    COUNT(*) AS customer_count
FROM customers
WHERE EXTRACT(YEAR FROM registration_date) BETWEEN 2018 AND 2024
GROUP BY registration_year
ORDER BY registration_year ASC;
```

Question 3b

Which shipping cities received more than 10 delivered orders in total? Display the city name and the count of delivered orders, ordered by count descending. **

Answer

```
SELECT
    shipping_city,
    COUNT(*) AS delivered_order_count
FROM orders
WHERE status = 'Delivered'
GROUP BY shipping_city
HAVING COUNT(*) > 10
ORDER BY delivered_order_count DESC;
```

Question 3c

Find all products whose total quantity sold across all order_items exceeds 50 units. Display product_id, product_name, and total quantity sold. Order by total quantity descending.

Answer

```
SELECT
    p.product_id,
    p.product_name,
    SUM(oi.quantity) AS total_qty_sold
```

```
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_id, p.product_name
HAVING SUM(oi.quantity) > 50
ORDER BY total_qty_sold DESC;
```

Question 3d

Show each employee's full name and the total number of orders they handled. Return only employees who handled 20 or more orders. Order by order count descending.

Answer

```
SELECT
    e.first_name || ' ' || e.last_name AS full_name,
    COUNT(o.order_id) AS order_count
FROM orders o
JOIN employees e ON o.employee_id = e.employee_id
GROUP BY e.employee_id, e.first_name, e.last_name
HAVING COUNT(o.order_id) >= 20
ORDER BY order_count DESC;
```

Question 3e

For each year in the dataset (2021–2024), show the total number of orders placed and the count of distinct customers who ordered that year. Order by year ascending.

Answer

```
SELECT
    EXTRACT(YEAR FROM order_date) AS order_year,
    COUNT(*) AS total_orders, COUNT(DISTINCT customer_id) AS distinct_customers
FROM orders
WHERE EXTRACT(YEAR FROM order_date) BETWEEN 2021 AND 2024
GROUP BY order_year
ORDER BY order_year ASC
```

Section D — LIKE & ILIKE

Topics: LIKE · ILIKE · % wildcard · _wildcard

Question 4a

Retrieve the first_name, last_name, and email of all customers whose email address ends with @gmail.com. Order alphabetically by last_name.

Answer

```
SELECT first_name, last_name, email
FROM customers
WHERE email LIKE '%@gmail.com'
ORDER BY last_name ASC;
```

Question 4b

List all products whose names include the word "set" anywhere, regardless of case. Display product_name, category_id, and unit_price, ordered by unit_price descending.

Answer

```
SELECT product_name, category_id, unit_price
FROM products
WHERE product_name ILIKE '%set%'
ORDER BY unit_price DESC;
```

Question 4c

Find all customers whose last name begins with the letters 'Ad' (case-insensitive). Display full name, city, and registration_date.

Answer

```
SELECT
    first_name || ' ' || last_name AS full_name,
    city,
    registration_date
FROM customers
WHERE last_name ILIKE 'Ad%';
```

Question 4d

Retrieve all products whose names contain "combo", "kit", or "pack" anywhere in the name (case-insensitive). Display product_name, category_id, and unit_price.

Answer

```
SELECT product_name, category_id, unit_price
FROM products
WHERE
    product_name ILIKE '%combo%'
```

OR product_name ILIKE '%kit%'
OR product_name ILIKE '%pack%';

Question 4e

Find all customers whose city name contains the letter sequence 'an' (case-insensitive). Display first_name, last_name, and city. Order by city, then last_name.

Answer

```
SELECT first_name, last_name, city
FROM customers
WHERE city ILIKE '%an%'
ORDER BY city ASC, last_name ASC;
```

Section E — JOINS

Topics: INNER JOIN, LEFT JOIN, Multi-table JOIN

Question 5a

Display the 50 most recent orders. For each, show: order_id, customer's full name, handling employee's full name, order_date, status, and shipping_city. Use INNER JOINS. Order by order_date descending.

Answer

```
SELECT
  o.order_id,
  c.first_name || ' ' || c.last_name AS customer_name,
  e.first_name || ' ' || e.last_name AS employee_name,
  o.order_date,
  o.status,
  o.shipping_city
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN employees e ON o.employee_id = e.employee_id
ORDER BY o.order_date DESC
LIMIT 50;
```

Question 5b

List all 800 customers alongside the total number of orders they have placed. Customers who have never ordered should show 0. Display customer_id, full name, city, and order_count. Order by order_count descending, then last_name ascending.

Answer

```
SELECT
  c.customer_id,
  c.first_name || ' ' || c.last_name AS full_name,
  c.city,
  COUNT(o.order_id) AS order_count
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name, c.city
ORDER BY order_count DESC, c.last_name ASC;
```

Question 5c

Produce a detailed order line report containing every row in order_items. Show order_id, order_date, customer full name, product_name, quantity, unit_price, discount, and a calculated line_total. Order by order_id ascending, then product_name ascending.

Answer

```
SELECT
  o.order_id,
  o.order_date,
  c.first_name || ' ' || c.last_name AS customer_name,
  p.product_name,
  oi.quantity,
  oi.unit_price,
  oi.discount,
  ROUND(oi.quantity * oi.unit_price * (1 - oi.discount / 100.0), 2) AS line_total
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
JOIN customers c ON o.customer_id = c.customer_id
JOIN products p ON oi.product_id = p.product_id
ORDER BY o.order_id ASC, p.product_name ASC;
```

Question 5d

Show all 35 employees with their full_name, role, region_name, and the total number of orders they have handled. Include employees with zero orders (show 0). Order by total orders descending, then last_name ascending.

Answer

```
SELECT
  e.first_name || ' ' || e.last_name AS full_name,
  e.role,
  r.region_name,
  COUNT(o.order_id) AS total_orders
FROM employees e
JOIN regions r    ON e.region_id = r.region_id
LEFT JOIN orders o ON e.employee_id = o.employee_id
GROUP BY e.employee_id, e.first_name, e.last_name, e.role, r.region_name
ORDER BY total_orders DESC, e.last_name ASC;
```

Question 5e

For each product category, list every product alongside the total number of distinct orders it has appeared in and the total quantity sold. Display category_name, product_name, times_ordered, and total_qty_sold. Order by category_name, then total_qty_sold descending.

Answer

```
SELECT
  c.category_name,
  p.product_name,
  COUNT(DISTINCT oi.order_id) AS times_ordered,
  SUM(oi.quantity) AS total_qty_sold
FROM categories c
JOIN products  p  ON c.category_id = p.category_id
LEFT JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY c.category_name, p.product_id, p.product_name
ORDER BY c.category_name ASC, total_qty_sold DESC;
```

Section F — CASE Expressions

Topics: CASE WHEN · CASE inside aggregates

Question 6a

Assign a price tier label to every product. Display product_name, category_name, unit_price, and price_tier. Order by unit_price ascending.

Answer

```
SELECT
  p.product_name,
  c.category_name,
  p.unit_price,
  CASE
    WHEN p.unit_price < 10000      THEN 'Budget'
    WHEN p.unit_price BETWEEN 10000 AND 99999 THEN 'Mid-Range'
    ELSE                          'Premium'
  END AS price_tier
FROM products p
JOIN categories c ON p.category_id = c.category_id
ORDER BY p.unit_price ASC;
```

Question 6b

Classify each of the 35 employees into a pay band based on their salary. Display full_name, role, salary, and pay_band. Order by salary descending.

Answer

```
SELECT
  first_name || ' ' || last_name AS full_name,
  role,
  salary,
  CASE
    WHEN salary >= 100000      THEN 'Executive'
    WHEN salary BETWEEN 80000 AND 99999 THEN 'Senior'
    ELSE                      'Entry Level'
  END AS pay_band
FROM employees
ORDER BY salary DESC;
```

Question 6c

For each order, calculate the total order value and classify it. Display order_id, order_date, status, total_order_value (rounded to 2 dp), and value_category. Order by total_order_value descending.

Answer

```

SELECT
    o.order_id,
    o.order_date,
    o.status,
    ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount / 100.0)), 2) AS total_order_value,
    CASE
        WHEN SUM(oi.quantity * oi.unit_price * (1 - oi.discount / 100.0)) > 500000 THEN 'High Value'
        WHEN SUM(oi.quantity * oi.unit_price * (1 - oi.discount / 100.0)) >= 100000 THEN 'Medium Value'
        ELSE 'Low Value'
    END AS value_category
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY o.order_id, o.order_date, o.status
ORDER BY total_order_value DESC;

```

Question 6d

Using CASE inside an aggregate, count how many products in each category fall into each price tier. Display category_name, budget_count, mid_range_count, premium_count.

Answer

```

SELECT
    c.category_name,
    COUNT(CASE WHEN p.unit_price < 10000 THEN 1 END) AS budget_count,
    COUNT(CASE WHEN p.unit_price BETWEEN 10000 AND 99999 THEN 1 END) AS mid_range_count,
    COUNT(CASE WHEN p.unit_price >= 100000 THEN 1 END) AS premium_count
FROM products p
JOIN categories c ON p.category_id = c.category_id
GROUP BY c.category_name
ORDER BY c.category_name;

```

Section G — Subqueries

Topics: Scalar · IN / NOT IN · NOT EXISTS · Derived tables

Question 7a

Find all products whose unit_price is above the average unit price of all products. Display product_name, category_id, and unit_price. Order by unit_price descending.

Answer

```
SELECT product_name, category_id, unit_price
FROM products
WHERE unit_price > (SELECT AVG(unit_price) FROM products)
ORDER BY unit_price DESC;
```

Question 7b

List all customers who have placed at least one order. Display their full name and city. Use a subquery with IN — do not use a JOIN.

Answer

```
SELECT
    first_name || ' ' || last_name AS full_name,
    city
FROM customers
WHERE customer_id IN (
    SELECT DISTINCT customer_id FROM orders
);
```

Question 7c

Find all products that have never appeared in any order. Display product_id, product_name, category_id, and unit_price.

Answer

```
SELECT product_id, product_name, category_id, unit_price
FROM products
WHERE NOT EXISTS (
    SELECT 1
    FROM order_items oi
    WHERE oi.product_id = products.product_id
);
```

Question 7d

Using subqueries, find the top 5 customers by total lifetime revenue. Display their full name, city, and total lifetime revenue rounded to 2 decimal places. Do not use a CTE.

Answer

```
SELECT
    c.first_name || ' ' || c.last_name AS full_name,
    c.city,
```

```

    revenue_data.total_lifetime_revenue
FROM customers c
JOIN (
    SELECT
        o.customer_id,
        ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount / 100.0)), 2) AS
total_lifetime_revenue
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    GROUP BY o.customer_id
) AS revenue_data ON c.customer_id = revenue_data.customer_id
ORDER BY revenue_data.total_lifetime_revenue DESC
LIMIT 5;

```

Question 7e

Find all customers whose total lifetime revenue exceeds the average lifetime revenue across all ordering customers. Display their full name, city, and total revenue (rounded to 2 dp). Order by total revenue descending. Do not use a CTE.

Answer

```

SELECT
    c.first_name || ' ' || c.last_name AS full_name,
    c.city,
    cust_rev.total_revenue
FROM customers c
JOIN (
    SELECT
        o.customer_id,
        ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount / 100.0)), 2) AS total_revenue
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    GROUP BY o.customer_id
) AS cust_rev ON c.customer_id = cust_rev.customer_id
WHERE cust_rev.total_revenue > (
    SELECT AVG(customer_total)
    FROM (
        SELECT
            o2.customer_id,
            SUM(oi2.quantity * oi2.unit_price * (1 - oi2.discount / 100.0)) AS customer_total
        FROM orders o2

```

```

        JOIN order_items oi2 ON o2.order_id = oi2.order_id
        GROUP BY o2.customer_id
    ) AS avg_sub
)
ORDER BY cust_rev.total_revenue DESC;

```

Section H — CTEs (Common Table Expressions)

Topics: WITH clause · Single CTE · Chained CTEs · CTE + CASE

Question 8a

Using a single CTE, calculate the total revenue per customer. Return the top 10 customers by revenue. Display customer_id, full name, city, and total_revenue rounded to 2 dp.

Answer

```

WITH customer_revenue AS (
    SELECT
        o.customer_id,
        ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount / 100.0)), 2) AS total_revenue
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    GROUP BY o.customer_id
)
SELECT
    c.customer_id,
    c.first_name || ' ' || c.last_name AS full_name,
    c.city,
    cr.total_revenue
FROM customers c
JOIN customer_revenue cr ON c.customer_id = cr.customer_id
ORDER BY cr.total_revenue DESC
LIMIT 10;

```

Question 8b

Using a CTE, identify the single best-selling product (by total quantity sold) in each category. Display category_name, product_name, and total_qty_sold.

Answer

```

WITH product_qty AS (
    SELECT

```

```

        p.product_id,
        p.product_name,
        p.category_id,
        SUM(oi.quantity) AS total_qty_sold
    FROM products p
    JOIN order_items oi ON p.product_id = oi.product_id
    GROUP BY p.product_id, p.product_name, p.category_id
)
SELECT
    c.category_name,
    pq.product_name,
    pq.total_qty_sold
FROM product_qty pq
JOIN categories c ON pq.category_id = c.category_id
WHERE pq.total_qty_sold = (
    SELECT MAX(total_qty_sold)
    FROM product_qty pq2
    WHERE pq2.category_id = pq.category_id
)
ORDER BY c.category_name;

```

Question 8c

Using two chained CTEs, analyse monthly performance for 2023 only. Show month number, total revenue, and vs_average ('Above Average' or 'Below Average'). Order by month ascending.

Answer

```

WITH monthly_revenue AS (
    SELECT
        EXTRACT(MONTH FROM o.order_date) AS month_num,
        ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount / 100.0)), 2) AS total_revenue
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    WHERE EXTRACT(YEAR FROM o.order_date) = 2023
    GROUP BY month_num
),
avg_revenue AS (
    SELECT AVG(total_revenue) AS avg_monthly_revenue
    FROM monthly_revenue
)

```

```

SELECT
    mr.month_num,
    mr.total_revenue,
    CASE
        WHEN mr.total_revenue > ar.avg_monthly_revenue THEN 'Above Average'
        ELSE 'Below Average'
    END AS vs_average
FROM monthly_revenue mr
CROSS JOIN avg_revenue ar
ORDER BY mr.month_num ASC;

```

Question 8d

Using CTEs, produce a customer frequency segmentation report. Return one row per segment showing the segment label and customer_count. Order by customer_count descending.

Answer

```

WITH order_counts AS (
    SELECT
        c.customer_id,
        COUNT(o.order_id) AS orders_placed
    FROM customers c
    LEFT JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY c.customer_id
)
SELECT
    CASE
        WHEN orders_placed >= 8 THEN 'High Frequency'
        WHEN orders_placed BETWEEN 4 AND 7 THEN 'Regular'
        WHEN orders_placed BETWEEN 1 AND 3 THEN 'Occasional'
        ELSE 'Inactive'
    END AS segment,
    COUNT(*) AS customer_count
FROM order_counts
GROUP BY segment
ORDER BY customer_count DESC;

```

Question 8e

Using a CTE, compute the year-over-year total revenue from delivered orders for each year in the dataset (2021–2024). Display order_year and total_revenue (rounded to 2 dp). Order by year ascending.

Answer

```
WITH delivered_revenue AS (  
  SELECT  
    EXTRACT(YEAR FROM o.order_date) AS order_year,  
    ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount / 100.0)), 2) AS total_revenue  
  FROM orders o  
  JOIN order_items oi ON o.order_id = oi.order_id  
  WHERE o.status = 'Delivered'  
    AND EXTRACT(YEAR FROM o.order_date) BETWEEN 2021 AND 2024  
  GROUP BY order_year  
)  
SELECT order_year, total_revenue  
FROM delivered_revenue  
ORDER BY order_year ASC;
```

Section I — Capstone Challenge

Topics: CTEs + JOINS + GROUP BY + Aggregates + CASE — Multi-concept

Question 9 — Employee Sales Performance Report

A comprehensive Employee Sales Performance Dashboard for all delivered orders (1 Jan 2021 – 30 Jun 2024), using at least two CTEs.

Answer

```
-- CTE 1: Revenue per order (from order_items)  
WITH order_revenue AS (  
  SELECT  
    oi.order_id,  
    SUM(oi.quantity * oi.unit_price * (1 - oi.discount / 100.0)) AS order_total  
  FROM order_items oi  
  GROUP BY oi.order_id  
)  
  
-- CTE 2: Aggregate delivered order stats per employee  
employee_stats AS (  
  SELECT  
    e.employee_id,  
    e.first_name,  
    e.last_name,  
    EXTRACT(YEAR FROM o.order_date) AS order_year,  
    ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount / 100.0)), 2) AS total_revenue  
  FROM employees e  
  JOIN orders o ON e.employee_id = o.employee_id  
  JOIN order_items oi ON o.order_id = oi.order_id  
  WHERE o.status = 'Delivered'  
    AND EXTRACT(YEAR FROM o.order_date) BETWEEN 2021 AND 2024  
  GROUP BY e.employee_id, e.first_name, e.last_name, EXTRACT(YEAR FROM o.order_date)
```

```

SELECT
    o.employee_id,
    COUNT(o.order_id) AS total_delivered_orders,
    ROUND(SUM(orf.order_total), 2) AS total_revenue,
    ROUND(AVG(orf.order_total), 2) AS avg_order_value,
    ROUND(MAX(orf.order_total), 2) AS best_single_order
FROM orders o
JOIN order_revenue orf ON o.order_id = orf.order_id
WHERE o.status = 'Delivered'
    AND o.order_date BETWEEN '2021-01-01' AND '2024-06-30'
GROUP BY o.employee_id
)

-- Final SELECT: All 35 employees with COALESCE for zero-order employees
SELECT
    e.first_name || ' ' || e.last_name AS employee_name,
    e.role,
    r.region_name,
    COALESCE(es.total_delivered_orders, 0) AS total_delivered_orders,
    COALESCE(es.total_revenue, 0) AS total_revenue,
    COALESCE(es.avg_order_value, 0) AS avg_order_value,
    COALESCE(es.best_single_order, 0) AS best_single_order,
    CASE
        WHEN COALESCE(es.total_revenue, 0) > 5000000 THEN 'Elite'
        WHEN COALESCE(es.total_revenue, 0) BETWEEN 1000000 AND 5000000 THEN
'Strong'
        WHEN COALESCE(es.total_revenue, 0) BETWEEN 100000 AND 999999 THEN
'Developing'
        ELSE 'Inactive'
    END AS performance_band
FROM employees e
JOIN regions r ON e.region_id = r.region_id
LEFT JOIN employee_stats es ON e.employee_id = es.employee_id
ORDER BY total_revenue DESC, employee_name ASC;

```

Section J — Bonus Challenge

Topics: All concepts combined

Question 10 — Customer Lifetime Value Report

For every customer registered before 2024, produce a full customer lifetime value report using at least one CTE.

Answer

```

WITH customer_stats AS (
    SELECT
        c.customer_id,
        COUNT(o.order_id) AS total_orders,
        COUNT(CASE WHEN o.status = 'Delivered' THEN 1 END) AS
delivered_orders,
        COUNT(CASE WHEN o.status = 'Cancelled' THEN 1 END) AS
cancelled_orders,
        ROUND(
            COALESCE(
                SUM(
                    CASE WHEN o.status = 'Delivered'
                        THEN oi.quantity * oi.unit_price * (1 - oi.discount / 100.0)
                    END
                ), 0
            ), 2
        ) AS lifetime_revenue,
        ROUND(
            COALESCE(
                SUM(
                    CASE WHEN o.status = 'Delivered'
                        THEN oi.quantity * oi.unit_price * (1 - oi.discount / 100.0)
                    END
                ) / NULLIF(COUNT(CASE WHEN o.status = 'Delivered' THEN 1 END), 0),
            0
        ), 2
        ) AS avg_order_value
    FROM customers c
    LEFT JOIN orders o ON c.customer_id = o.customer_id
    LEFT JOIN order_items oi ON o.order_id = oi.order_id
    WHERE EXTRACT(YEAR FROM c.registration_date) < 2024
    GROUP BY c.customer_id
)

SELECT

```

```

c.first_name || ' ' || c.last_name      AS customer_name,
c.city,
EXTRACT(YEAR FROM c.registration_date)::INT  AS registration_year,
cs.total_orders,
cs.delivered_orders,
cs.cancelled_orders,
cs.lifetime_revenue,
cs.avg_order_value,
CASE
  WHEN cs.lifetime_revenue > 500000
    AND cs.delivered_orders >= 5          THEN 'VIP'
  WHEN (cs.lifetime_revenue BETWEEN 100000 AND 500000)
    OR (cs.delivered_orders BETWEEN 2 AND 4)      THEN 'Loyal'
  WHEN cs.delivered_orders = 1                THEN 'One-Time Buyer'
  WHEN cs.delivered_orders = 0 AND cs.total_orders > 0      THEN 'No Conversions'
  ELSE                                          'Inactive'
END AS customer_segment
FROM customers c
JOIN customer_stats cs ON c.customer_id = cs.customer_id
ORDER BY cs.lifetime_revenue DESC, customer_name ASC;

```